

# PPL LAB-9 EXERCISES

📅 Date @March 27, 2026

## PROGRAMS ON MATRIX IN CUDA - PART 2

NAME : DEVADATHAN N R

ROLL NO : 04

SECTION : CSE A1

REG NO : 230905010

### Question 1:

Write a program in CUDA to perform parallel Sparse Matrix - Vector multiplication using compressed sparse row (CSR) storage format. Represent the input sparse matrix in CSR format in the host code

#### Code:

```
#include <cuda_runtime.h>
#include <stdio.h>
#include <stdlib.h>

__global__ void csr_kernel(int num_rows, int *row_ptr, int *col_index, int *data, int *x,
int *y) {
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < num_rows) {
        int dot = 0;
        int row_start = row_ptr[row];
        int row_end = row_ptr[row + 1];
        for (int i = row_start; i < row_end; i++) {
            dot += data[i] * x[col_index[i]];
        }
        y[row] = dot;
    }
}

int main(){
    int num_rows, num_cols;
    printf("Enter number of rows and columns: ");
    scanf("%d %d", &num_rows, &num_cols);
    int matrix[num_rows][num_cols];
    int num_non_zeros = 0;
    printf("Enter the sparse matrix (%dx%d):\n", num_rows, num_cols);
    for (int i = 0; i < num_rows; i++) {
        for (int j = 0; j < num_cols; j++) {
            scanf("%d", &matrix[i][j]);
            if (matrix[i][j] != 0) num_non_zeros++;
        }
    }
}
```

```

int data[num_non_zeros];
int col_index[num_non_zeros];
int row_ptr[num_rows + 1];

// Convert matrix to CSR format
row_ptr[0] = 0;
int idx = 0;
for (int i = 0; i < num_rows; i++) {
    for (int j = 0; j < num_cols; j++) {
        if (matrix[i][j] != 0) {
            data[idx] = matrix[i][j];
            col_index[idx] = j;
            idx++;
        }
    }
    row_ptr[i + 1] = idx;
}
int x[num_cols];
printf("Enter vector x (%d values):\n", num_cols);
for (int i = 0; i < num_cols; i++) scanf("%d", &x[i]);
int y[num_rows];
int *d_row_ptr, *d_col_index;
int *d_data, *d_x, *d_y;
cudaMalloc((void**)&d_row_ptr, (num_rows + 1) * sizeof(int));
cudaMalloc((void**)&d_col_index, num_non_zeros * sizeof(int));
cudaMalloc((void**)&d_data, num_non_zeros * sizeof(int));
cudaMalloc((void**)&d_x, num_cols * sizeof(int));
cudaMalloc((void**)&d_y, num_rows * sizeof(int));
cudaMemcpy(d_row_ptr, row_ptr, (num_rows + 1) * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_col_index, col_index, num_non_zeros * sizeof(int), cudaMemcpyHostToDevice);
e);
cudaMemcpy(d_data, data, num_non_zeros * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_x, x, num_cols * sizeof(int), cudaMemcpyHostToDevice);
int block_size = 256;
int grid_size = (num_rows + block_size - 1) / block_size;
csr_kernel<<<grid_size, block_size>>>(num_rows, d_row_ptr, d_col_index, d_data, d_x,
d_y);
cudaMemcpy(y, d_y, num_rows * sizeof(int), cudaMemcpyDeviceToHost);

printf("CSR Representation:\n");
printf("Data Array: ");
for (int i = 0; i < num_non_zeros; i++) printf("%d ", data[i]);
printf("\nCol_index Array: ");
for (int i = 0; i < num_non_zeros; i++) printf("%d ", col_index[i]);
printf("\nRow_ptr Array: ");
for (int i = 0; i <= num_rows; i++) printf("%d ", row_ptr[i]);
printf("\n");
printf("\nResult vector y:\n");
for (int i = 0; i < num_rows; i++)printf("%d\n", y[i]);

cudaFree(d_row_ptr);

```

```

    cudaFree(d_col_index);
    cudaFree(d_data);
    cudaFree(d_x);
    cudaFree(d_y);
    return 0;
}

```

### Output:

```

STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/09_Lab$ nvcc l9q1.cu -o l9q1
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/09_Lab$ ./l9q1
Enter number of rows and columns:
3 3
Enter the sparse matrix (3x3):
10 0 20
30 40 50
0 0 60
Enter vector x (3 values):
1 2 3
CSR Representation:
Data Array: 10 20 30 40 50 60
Col_index Array: 0 2 0 1 2 2
Row_ptr Array: 0 2 5 6

Result vector y:
70
260
180

```

### Question 2:

Write a program in CUDA to read M X N matrix A and replace 1st row of this matrix by same elements, 2nd row elements by square of each element and 3rd row elements by cube of each element and so on.

### Code:

```

#include <stdio.h>
#include <cuda_runtime.h>
#include <math.h>

__global__ void transformMatrix(int *A, int M, int N) {
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < M) {
        for (int col=0; col<N; col++) {
            A[row * N + col] = (int)roundf(pow((float)A[row * N + col], (float)(row +
1)));
        }
    }
}

int main() {
    int M, N;
    printf("Enter M and N: ");
    scanf("%d %d", &M, &N);
    int size = M * N * sizeof(int);

```

```

int *h_A = (int*)malloc(size);
printf("Enter matrix elements:\n");
for (int i = 0; i < M * N; i++) {
    scanf("%d", &h_A[i]);
}
int *d_A;
cudaMalloc(&d_A, size);
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);

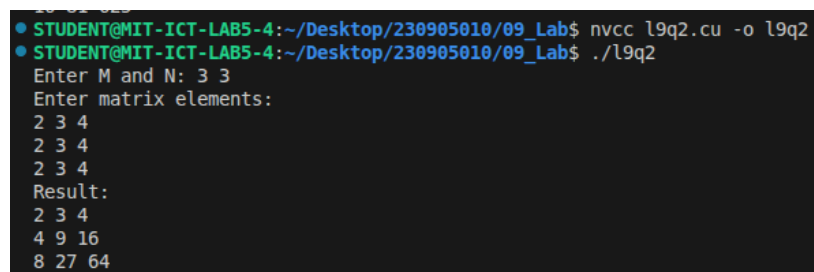
// Updated grid and block dimensions to handle larger M
int threadsPerBlock = 256;
int blocksPerGrid = (M + threadsPerBlock - 1) / threadsPerBlock;

transformMatrix<<<blocksPerGrid, threadsPerBlock>>>(d_A, M, N);

cudaMemcpy(h_A, d_A, size, cudaMemcpyDeviceToHost);
printf("Result:\n");
for (int i = 0; i < M; i++) {
    for (int j = 0; j < N; j++) {
        printf("%d ", h_A[i * N + j]);
    }
    printf("\n");
}
free(h_A);
cudaFree(d_A);
return 0;
}

```

### Output:



```

STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/09_Lab$ nvcc l9q2.cu -o l9q2
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/09_Lab$ ./l9q2
Enter M and N: 3 3
Enter matrix elements:
2 3 4
2 3 4
2 3 4
Result:
2 3 4
4 9 16
8 27 64

```

### Question 3:

Write a CUDA program that reads a matrix A of size M X N and produce an output matrix B of the same size such that it replaces all the non border elements (numbers in bold) of A with its equivalent 1's complement and remaining elements same as matrix A

### Code:

```

#include<stdio.h>
#include <cuda_runtime.h>

```

```

__global__ void processMatrix(int *A, int *B, int M, int N) {
    int row = threadIdx.y;
    int col = threadIdx.x;
    if (row < M && col < N) {
        int index = row * N + col;
        if (row == 0 || row == M - 1 || col == 0 || col == N - 1) {
            B[index] = A[index];
        } else {
            B[index] = ~A[index];
        }
    }
}

void printBinary(int num) {
    for (int i = 2; i >= 0; i--) {
        printf("%d", (num >> i) & 1);
    }
}

int main() {
    int M, N;
    printf("Enter M and N: ");
    scanf("%d %d", &M, &N);

    int size = M * N * sizeof(int);
    int *h_A = (int *)malloc(size);
    int *h_B = (int *)malloc(size);

    printf("Enter matrix elements:\n");
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &h_A[i * N + j]);
        }
    }

    int *d_A, *d_B;
    cudaMalloc((void **)&d_A, size);
    cudaMalloc((void **)&d_B, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    dim3 threadsPerBlock(N, M);
    processMatrix<<<1, threadsPerBlock>>>(d_A, d_B, M, N);
    cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceToHost);

    printf("Result (in binary):\n");
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            if (i == 0 || i == M - 1 || j == 0 || j == N - 1) printf("%d", h_B[i * N +
j]);
            else printBinary(h_B[i * N + j]);
            printf(" ");
        }
    }
}

```

```

    }
    printf("\n");
}

free(h_A);
free(h_B);
cudaFree(d_A);
cudaFree(d_B);

return 0;
}

```

### Output:

```

STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/09_Lab$ nvcc l9q3.cu -o l9q3
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/09_Lab$ ./l9q3
Enter M and N: 4 4
Enter matrix elements:
1 2 3 4
6 5 8 3
2 4 10 1
9 1 2 5
Result (in binary):
1 2 3 4
6 010 111 3
2 011 101 1
9 1 2 5

```